绪论

qwaszx

2026 年 7 月 29 日

今天的内容

- 为什么学 OI / 学 OI 有什么好处?
- 怎么学 OI?

为什么学了 OI

你为什么学了 OI?

为什么学了 OI

你为什么学了 OI?
我为什么学了 OI。

学 OI 的好处

- 提升逻辑思维能力
- 算法思维
  - 如何描述解决问题的步骤
  - 对计算的理解
- 计算机和离散数学知识
- OI 社群
- 更多地玩电脑
- 升学与就业

升学与就业

升学方面：

- 金牌可以保送清华姚班/北大图灵班
- 银牌可以破格入围强基计划
- 省一对强基计划和各种自主招生有一定帮助

就业方面：

- OI/XCPC 竞赛获奖对于计算机相关行业有一定帮助
- 可以继续做教培，相对普通文化课报酬更高
- 对口 TCS（理论计算机科学）

好处说完了，那么坏处呢

- 机会成本：选择 OI 需要放弃一些其他东西
- 更多地使用计算机和互联网，可能会损害身心健康。
- 对于升学和就业的性价比不算很高
  - 目前，银牌相比省一的优势不大，但是困难得多
  - 集训队每年只有 50 人，竞争激烈

总结

初中阶段，适合学文化课有余力并且学 OI 学得开心的同学学习。
高中阶段，功利角度建议根据自身成绩谨慎选择（高一进省队 -> 高二进集训队）
兴趣是第一位的！

OI 比赛的级别

1. CSP-J
2. CSP-S
3. NOIP
4. 省选
5. NOI
6. CTS
7. IOI

其中 CSP-J 到 CSP-S 属于前期，CSP-S 到 NOIP 属于中期，省选到 NOI 属于后期，再往后是大后期

做题的过程

1. **读题**
2. **分析性质**
3. **抽象问题**
4. **使用/改编/设计算法来处理**
5. **实现**

前期最重要的是 1 和 5，中期最重要的是 3 和 4，后期和大后期最重要的是 2 和 4
学习的过程中，每个部分都在逐步提高

读题与实现

抽象/封装：忽略某个东西的内部结构，只关注其输入和输出
在做题中练习
多看看其他人的写法

抽象问题，调用算法

我们会学习很多的算法。学习的过程中，

- 关注算法整体的功能，熟知一个问题可以被什么算法解决，一个算法可以解决什么问题，有哪些变种和扩展
- 关注算法的每一步，熟知其原理、作用，多思考可以如何扩展
- 关注证明

在做题中练习
多总结

分析性质，设计算法

也就是所谓的人类智慧。一般来说，很难！
多总结题目和算法的模型。相似的模型往往具有相似的性质。
多思考题解/算法的每一步是如何想到的。
尝试建立不同概念和不同算法之间的联系，抽象其共性

训练与学习资源

OJ/题库：洛谷、QOJ、LOJ、UOJ；Codeforces、Atcoder
比赛：Codeforces、Atcoder
学习资源：oiwiki、各种优质博客（如 liu_cheng_ao 推荐的、command_block、Elegia）
各种 AI
OI 社群与搜索引擎。不懂就问/搜！

训练的方法

- 学习知识点
- 刷知识点对应的题
- 对知识点进行归纳总结
- 通过模拟赛进行不同知识点的综合练习/复习

刷题的方法

先自行思考，思考到自己一直没有任何想法为止，然后看题解。这个时间长短是不定的
看题解的过程中也要进行思考：作者说的有没有道理，作者是怎么想到的，我能不能自行补全剩下的部分，哪些部分还可以优化
想出做法/看懂题解后进行实现，这个过程中不要参考任何现成代码
做完题后看看题解区有没有不同做法，看看其他人的代码都是怎么写的，想想自己的代码哪些地方能更快/更好写
看看题目是否是**经典问题**，是则需要在归纳总结时**记录**

经典问题

有一些问题是经典的：在拆掉包装后，其技巧/模型非常泛用。每做完一道题都应该思考这道题是不是这种问题。换句话说，做题后要努力将题目的模型/技巧总结出来。

刷什么题

学习资源中提到的题、自行根据知识点搜索相关题目（如洛谷题单）
重要的是刷题前中后的**思考**而不是刷题本身。所以，非常同质化的题不用多刷，几道就够了（再多的可以口胡）
刷题可以形成题目 <-> 模型 <-> 算法的条件反射，或者说直觉。

训练的理念

循序渐进，避免一直学太简单/太难的东西
青少年的智力、理解能力会随着年龄提高
熟练掌握 OI 大部分知识点通常需要且只需要几千个小时，相当于 1 到 2 年 *8h（高中脱产/初中或者小学就开始学）

OI 之外

对初中生来说，我非常建议学好文化课，义务教育是必要的。不过可以多和家长老师沟通，放弃重复性过强的文化课练习。OI 以兴趣为主，无需强求。
OI 涉及大量的思考，所以充足睡眠非常重要，建议想办法得到充足睡眠。最起码的原则是不能一天比一天困。
多锻炼。一方面大学也有各种体育测试（如 3km），另一方面人生是场马拉松，健康地活着才是成功
人际交流，线上线下均可

代码能力

可以用代码实现想法
知道一个东西能不能写、大概是怎么写的、好不好写
不出 bug 地尽量快地写出来
学习更优秀的写法（更好写/更快）

先想再写 vs 边写边想

我的习惯：在写某个部分时，对这个部分的整体结构必须提前想好，细节可以边写边想

抽象/封装的代码设计理念

目前你需要非常熟练的东西

- 顺序、分支、循环结构的程序设计
- 各种变量类型、结构体、数组
- 函数
- 简单的模拟、枚举

如果不熟练，可以先多做做橙题

总结

- 对 OI 概况、利弊的了解
- OI 的一些学习方法

最后

希望大家学得开心

最后

希望大家学得开心

谢谢大家！